

Project Executable Files

Date	08 OCTOBER 2026
Team ID	UNM1611GUFRAN
Project Name	AI FAQ Assistant API
Maximum Marks	3 Marks

Project Executable Files

This section requires submission of all executable and deployable files for your project. Ensure all files are properly organized, named, and uploaded to the specified submission platform. The evaluator must be able to run or deploy your solution using the provided files and instructions.

Step 1: Submission Checklist

S.No	Item to Submit	Submitted (Yes / No / NA)
1	Complete source code (all files and folders)	Yes
2	README / Setup Guide (instructions to run the project)	Yes
3	requirements.txt / package.json / dependency file	Yes
4	Database schema / seed files (if applicable)	Yes
5	Environment configuration file (.env.example or similar)	Yes
6	Deployed application URL (if hosted)	NA – Local Development Project

7	APK / Executable binary (if applicable for mobile/desktop apps)	NA – Backend REST API Project
8	Dockerfile / Containerization config (if applicable)	NA
9	Test files and test results	Yes
10	Demo video or walkthrough (if required)	NA

Step 2: File / Folder Structure

AI_FAQ_Assistant_API/

```
|
|  └─ src/
|    |  └─ config/
|    |    └─ db.js
|    |
|    |  └─ controllers/
|    |    └─ aiController.js
|    |    └─ authController.js
|    |    └─ faqController.js
|    |
|    |  └─ middleware/
|    |    └─ authMiddleware.js
|    |    └─ errorMiddleware.js
|    |    └─ validationMiddleware.js
|    |
|    |  └─ models/
|    |    └─ FAQ.js
|    |    └─ User.js
|    |
|    |  └─ routes/
|    |    └─ aiRoutes.js
|    |    └─ authRoutes.js
|    |    └─ faqRoutes.js
|    |
|    |  └─ services/
|    |    └─ geminiService.js
|    |
|    |  └─ utils/
|    |    └─ helpers.js
|    |
|    └─ app.js
|    └─ server.js
|
|  └─ .env
|  └─ .env.example
|  └─ package.json
|  └─ package-lock.json
|  └─ README.md
```

Step 3: Deployment / Access Details

Field	Details
Hosted / Deployed URL	NA – Project runs locally on http://localhost:5000
Login Credentials (Demo)	Register a new user using the /api/auth/register endpoint
Platform / Hosting Provider	Local Node.js Development Environment
Repository Link	NA – Not publicly hosted
Demo Video Link	NA

Step 4: Run Instructions

Step 1: Install Node.js

Install **Node.js 18+ or Node.js 20+** on the system.

Verify the installation:

```
node --version npm
```

```
--version
```

Step 2: Install MongoDB

Install and start **MongoDB Community Server**, or use a MongoDB Atlas connection string.

The default local database connection is: mongodb://127.0.0.1:27017/ai_faq_assistant

Step 3: Open the Project

Open the **AI FAQ Assistant API** project folder in Visual Studio Code.

Open the terminal in the project root directory.

Step 4: Install Dependencies Run:

```
npm install
```

Step 5: Configure Environment Variables Create

a .env file in the project root:

```
PORT=5000
```

```
MONGO_URI=mongodb://127.0.0.1:27017/ai_faq_assistant
```

```
JWT_SECRET=your_jwt_secret_key
```

```
GEMINI_API_KEY=your_google_gemini_api_key
```

Replace the Gemini API key with a valid Google Gemini API key.

Step 6: Start the Server For

development mode:

npm run dev Or:

npm start

Step 7: Verify Server

The server should run at: <http://localhost:5000>

The MongoDB connection should also be established successfully.

Step 8: Open Postman Import:

AI_FAQ_Assistant_API.postman_collection.json

into Postman. **Step 9: Register a User Send:**

POST /api/auth/register with:

```
{  
  "name": "John Doe",  
  "email": "john@gmail.com",  
  "password": "123456"  
}
```

Step 10: Login Send:

POST /api/auth/login

with:

```
{  
  "email": "john@gmail.com",  
  "password": "123456"  
}
```

The API returns a JWT token.

Step 11: Test Protected APIs Use

the JWT token in: Authorization:

Bearer <token> Test the

following:

GET /api/auth/profile

POST /api/faqs

PUT /api/faqs/:id

DELETE /api/faqs/:id

POST /api/ai/answer

POST /api/ai/generate-faq

Step 12: Test Public APIs

Test:

GET /api/faqs

GET /api/faqs/:id

GET /api/faqs/search?q=MongoDB

Step 5: Known Issues / Limitations

S.No	Known Issue / Limitation	Workaround / Status
1	Gemini AI services require a valid Google Gemini API key and internet connection.	Configure a valid GEMINI_API_KEY in the .env file.
2	The current project is configured for local MongoDB and local server execution.	Start MongoDB locally before running the Node.js server, or configure MongoDB Atlas.
3	AI-generated responses may take longer than normal FAQ/database operations because they depend on the external Gemini API.	Allow sufficient response time and ensure a stable internet connection.